

Module : Evaluation De Performances Master 2 Calcul Haute Performance et Simulation (CHPS)

Soraya Zertal

Li-PaRAD-Université de ParisSaclay-UVSQ

Intervenants

- Soraya Zertal, Laboratoire Li-PaRAD, UPSacaly-UVSQ
`soraya.zertal@uvsq.fr`
- Sebastien Gougeaud, CEA
`sebastien.gougeaud@cea.fr`

Organisation et évaluation

- **Organisation de l'enseignement :**
Cours, travaux dirigés/pratiques, analyse d'article scientifique et présentation (soutenance)
- **Evaluation :**
Contrôle continu : TP, présentation orale, évaluation écrite

Éléments de bibliographie

- “The art of computer systems Performance Analysis: Techniques for Experimental design, Measurement, Simulation and Modeling”, Raj Jain, Wiley Edition.
- “Handbook of Simulation: Principles, Methodology, Advances, Applications, and Practice”, Jerry Banks, Wiley Edition.
- “Queueing Systems” Leonard Kleinrock, Wiley Edition.

Présentation générale du module

- Introduction à l'Evaluation Des Performances : concepts et application au HPC
- Méthodes et techniques d'évaluation
 - Workloads
 - Modélisation mathématique
 - Simulation
 - Mesure

Plan :

Concepts de base :

- 1 Motivation et problématique
- 2 Approche à suivre
- 3 Checklist pour éviter les erreurs
- 4 Métriques de base à évaluer
- 5 Caractéristiques d'un bon métrique
- 6 Evaluer les performances : méthodes à adopter

Plan :

Evaluation des performances pour le HPC :

- 1 Objectifs
- 2 Parallélisme et performance
- 3 Métriques classiques dans le HPC
- 4 Modèles et lacunes
- 5 Evaluation

Motivation et problématique

But de tout système : fournir le meilleur rapport
coût/**performance**

⇒ Nécessité d'évaluer cette **performance** à toutes les étapes
du cycle de vie d'un système :

- 1 Conception : prédiction
- 2 Réalisation : choix matériel ou logiciel
- 3 Commercialisation : mise en valeur de métriques
- 4 Utilisation : tuning selon le contexte
- 5 Maintenance et Mise à jour : contrôle (monitoring)

Motivations et problématique

Objectifs de l'évaluation de performances :

- Mieux connaître la **faisabilité** d'un système/application et l'**intérêt** de son développement/codage.
- Détecter et mieux comprendre les **limitations** d'un système.
- Bien cibler les **améliorations** possibles : ajout de ressources (exp. processeurs), réorganisation des traitements, réorganisation des charges,...etc

Approche à suivre

L'approche générale d'évaluation suit les étapes suivantes:

- Définir le système à analyser et préciser le but à atteindre.
- Etablir la liste des services à fournir par le système analysé.
- Sélectionner les métriques relatifs à la vitesse, la précision et la disponibilité du système analysé.

Approche à suivre

- Etablir **la liste des paramètres** affectant les performances du système analysé.
A distinguer les paramètres liés au système de ceux liés aux types de workloads.
- **Sélectionner les facteurs** (parmi les paramètres) variables pendant l'évaluation ainsi que **les valeurs** qu'ils peuvent prendre.
- **Sélectionner la méthode** d'évaluation appropriée parmi la modélisation mathématique, la simulation ou la mesure.

Approche à suivre

- Sélectionner les charges auxquelles sera soumis le système à évaluer : **synthétiques** ou **réelles**.
- Concevoir les jeux de tests pouvant fournir le maximum d'informations pour l'étude du comportement du système.
- Analyser et interpréter les données collectées à l'issue des tests en utilisant des **techniques statistiques**.
- Présenter les résultats et les communiquer sous une forme claire et compréhensible, souvent sous forme graphique, en utilisant une échelle appropriée.

Checklist pour éviter les erreurs (R. Jain)-Annexe

- Is the system correctly defined and the goals clearly stated?
- Are the goals stated in an unbiased manner?
- Have all the steps of the analysis followed systematically?
- Is the problem clearly understood before analyzing it?
- Are the performance metrics relevant for this problem?
- Is the workload correct for this problem?
- Is the evaluation technique appropriate?
- Is the list of parameters that affect performance complete?

Checklist pour éviter les erreurs (R. Jain)

- Have all parameters that affect performance been chosen as factors to be varied?
- Is the experimental design efficient in terms of time and results?
- Is the level of detail proper?
- Is the measured data presented with analysis and interpretation?
- Is the analysis statistically correct?
- Has the sensitivity analysis been done?
- Would errors in the input cause an insignificant change in the results?
- Have the outliers in the input/output been treated properly?

Checklist pour éviter les erreurs (R. Jain)

- Have the future changes in the system and workload been modeled?
- Has the variance of input been taken into account?
- Has the variance of the results been analyzed?
- Is the analysis easy to explain?
- Is the presentation style suitable for its audience?
- Have the results been presented graphically as much as possible?
- Are the assumptions and limitations of the analysis clearly documented?

Introduction aux métriques

Les **métriques communs** pour l'évaluation des performances :

- Temps de réponse
- Débit
- Accélération
- Efficacité
- Taux d'utilisation
- Fiabilité
- Disponibilité

Temps de réponse

Définition :

Le temps séparant la requête de l'utilisateur de la réponse du système. Le temps de réponse augmente avec la charge du système sous l'effet de l'augmentation du temps d'attente.

Temps de réponse - stretch factor

Le **facteur d'extension** (stretch factor) est le rapport du temps de réponse pour une certaine charge à celui pour une charge minimale.

Le **facteur d'extension** quantifie l'**impact de l'augmentation de la charge sur l'allongement du temps de réponse**.

Débit

Définition :

Taux de requêtes exécutées en une unité de temps.

Système	Unité de débit
Interactif	requêtes/s
CPU	MIPS ou MFLOPS
Batch	job/s
Réseau	packet-bits/s (pps,bps)
Transactionnel	transactions/s (TPS)

Débit

Remarque :

Le **débit** augmente avec la charge jusqu'à une certaine valeur maximale, une sorte de borne supérieure (exple: bande passante en réseau).

Débit

Débit/capacité nominal est le débit maximal pouvant être délivré par le système soumis à une charge idéale.

Exemple :

Après le start-up, une instruction est exécutée par le pipeline à chaque cycle s'il n'y a aucune dépendance pénalisante ou branchement.

Débit/capacité d'usage est le débit maximal pouvant être obtenu tout en respectant la limite imposée au temps de réponse pour le maintenir en deçà d'une certaine valeur acceptable par l'utilisateur.

Débit - MIPS et MFLOPS

MIPS: Millions d'instructions par seconde

$$MIPS = \frac{nb \text{ instrs}}{10^6 \times tps \text{ execution}}$$

MFLOPS: Millions d'instructions flottantes par seconde
Donc, c'est le MIPS restreint aux instructions flottantes.

Remarques : Pour les deux unités, on considère que les instructions sont en mémoire. De même, toutes les deux (surtout le MIPS) ne constituent pas de bons indicateurs de performances.

Accélération

Définition :

Le rapport entre le temps séquentiel et le temps parallèle (amélioré de manière générale)

$$Acc = Speedup = \frac{Tps_{\text{sequentiel}}}{Tps_{\text{parallele}}}$$

Le calcul de l'accélération par La **loi d'Amdahl** simple:

$$Acc = Speedup = \frac{1}{(1 - F_a) + \frac{F_a}{P}}$$

F_a : fraction parallélisable (améliorée) du code

P : nombre de processeurs utilisés

Accélération/coût

L'Accélération n'est **pas linéaire** en fonction du nombre de processeurs. Il y a des coûts supplémentaires liés à la parallélisation (amélioration) tels que : les échanges de messages, la synchronisation ...etc

Le calcul de l'accélération par la **loi d'amdahl enrichie** :

$$Acc = \frac{1}{(1-f_a) + (\frac{f_a}{P}) + Ct(f_a, P)}$$

$Ct(f_a, P)$: Coût lié à la parallélisation de la fraction f_a du code sur P processeurs.

Efficacité

Définition :

L'Efficacité d'une accélération obtenue par l'utilisation de P processeurs se calcule par:

$$E = \frac{Acc}{P}$$

ou bien :

$$E = \frac{Tps_{\text{sequentiel}}}{Tps_{\text{parallele}} \times P}$$

Plus l'efficacité est proche de 1, plus l'accélération est optimale.

A contrario, $(1 - E)$ désigne la part de l'inefficacité due à la communication/synchronisation liée à la parallélisation.

Taux d'utilisation

Définition :

Fraction du temps pendant laquelle la ressource est utilisée.

Calcul :

$$\frac{\text{temps d'occupation de la ressource}}{\text{le temps total ecoule}}$$

Buts :

Maximiser le taux d'utilisation d'une ressource.

Harmoniser au mieux les différentes ressources $\Rightarrow$ l'équilibrage de charge (Load Balancing).

Efficacité et taux d'utilisation

Dans un environnement parallèle:

→ L'efficacité reflète le taux d'utilisation des ressources fonctionnant en parallèle.

→ Plus elle tend vers 1, plus le taux d'utilisation est meilleur.

Accélération/coût

Strong Scaling:

Pour une taille constante d'un problème n , on augmente le nombre de processeurs P et on observe la diminution du temps d'exécution en conséquence.

Une extensibilité forte (Strong Scaling) implique qu'on va p fois plus vite avec P processeurs sur le même code de taille n , l'efficacité tend alors vers 1.

Accélération/coût

**Mais la parallélisation peut avoir un autre but
que celui de la réduction du temps
d'exécution ...**

Affiner les modèles, préciser plus les résultats,

Accélération/coût

Weak Scaling:

Une extensibilité faible (Weak Scaling) consiste à faire augmenter la taille du problème n et le nombre de processeurs P ensemble.

Le but est de traiter des problèmes plus gros en maintenant le temps d'exécution constant.

Accélération/coût

Loi de Gustafson-Barsis :

La meilleure 'accélération obtenue en exécutant un code de taille n sur P processeurs, avec s la fraction de son temps temps d'exécution passé sur le code séquentiel, est :

$$Acc(n, P) \leq P + (1 - P)s$$

Alors que la loi d'Amdhal limite cette accélération à P

Fiabilité (Reliability)

Définition:

L'aptitude d'un système à effectuer ses fonctions, de **fournir des résultats non erronés** et de maintenir ce fonctionnement en toutes circonstances pendant un temps T .

Data Reliability : le système de stockage -moyennant une redondance intégrale ou partielle- garantit l'accès à des données non erronées.

Network Reliability : le réseau garantit que les messages sont délivrés aux destinataires

Fiabilité (Reliability)

La fiabilité se **mesure** généralement par la **probabilité d'arrivée des erreurs**.

Néanmoins, il est d'usage de considérer le temps moyen séparant l'arrivée des erreurs, le **Mean Time Between Errors (MTBE)**.

Disponibilité (Availability)

Définition :

C'est la période du temps pendant laquelle le système **peut servir les requêtes** utilisateur.

C'est la période **uptime** par opposition à celle **downtime** où le système ne répond plus.

La disponibilité se mesure généralement par le temps moyen avant la panne (**Mean Time To Failure -MTTF-**) ou le temps moyen entre les pannes (**Mean Time Between Failures -MTBF-**).

Caractéristique : linéarité

Le métrique doit indiquer une performance **linéairement proportionnelle** à la performance actuelle.

Les utilisateurs sont assez sensibles à cette caractéristique mais pas tous les métriques peuvent l'être. Certains métriques sont logarithmiques.

Caractéristique : fiabilité

Un métrique est considéré fiable, si les valeurs de ce métrique pour les systèmes A et B, indiquent que le système A est plus performant que le système B et le système A est effectivement **toujours** plus performant que le système B.

Cela paraît évident mais il ne l'est pas toujours.

Exemple : MIPS n'est pas un métrique fiable.

Caractéristique : fiabilité

Exemple de la non fiabilité du MIPS

Le benchmark whetstone est exécuté sur une machine pouvant utiliser un coprocesseur flottant. Une itération de ce benchmark dure 1.08s et génère 1.6 MIPS avec le coprocesseur mais dure 13.6s et génère 2.7 MIPS sans coprocesseur.

L'exécution en utilisant le coprocesseur est alors plus performante (12.6 fois plus rapide) pour un MIPS moindre. Ce dernier ne reflète donc pas cette performance.

Caractéristique : déterminisme

Un métrique est **déterministe** SSI il donne la **même valeur à chaque fois** que le test est effectué.

Caractéristique : facilité de mesure

Un métrique difficile à mesurer est par conséquent peu utilisé.

De plus, une mesure difficile à effectuer a des chances d'être erronée.

Méthodes d'évaluation des performances

Trois méthodes pour l'évaluation des performances: la **modélisation analytique**, la **simulation** et la **mesure**.

La sélection de la méthode appropriée repose en premier lieu sur l'étape dans laquelle se trouve le système à analyser au moment de l'analyse.

Aussi, le coût temporel et financier de l'évaluation, les outils disponibles ainsi que le degré de précision des résultats.

Méthodes d'évaluation des performances

Opter pour la **mesure** si le **système existe** et est disponible (ou un système s'y approchant puis extrapoler).

Autrement, opter pour la **simulation** ou la méthode analytique.

La **simulation** est plus **consommatrice en temps** mais nécessite moins de **simplifications** que la **modélisation analytique**.

En général, on utilise 2 méthodes et on compare leurs résultats pour validation.

Evaluation des performances pour le HPC : Plan

- Objectifs
- Parallélisme et performance
- Métriques classiques pour le HPC
- Modèles et lacunes
- Evaluer les performances

Performance en HPC : Objectifs

- Proche du matériel :
Prédire les performances en se basant sur les progrès technologiques, mesurer, caractériser, comparer des solutions HW (par modèle), proposer des évolutions ...
- Très orienté performances élémentaires :
 - Activité annexe: stress du matériel grâce aux conditions aux limites
 - Nécessite parfois le recours à l'assembleur pour maîtriser ce qu'on fait
 - Etudes en profondeur, nombre de benchmarks clients

Parallélisme et performance

- Performance HPC
Loi d'Amdahl, parallélisation(scale up,speed up),
workload, les métriques classiques mais aussi : taux de
miss, définition xxx-bound, latences et débit mémoire,
ordres de grandeur

Parallélisme et Amdahl

HPC = Parallèle ou massivement parallèle

Que peut on paralléliser ?

$$\text{Temps total} = \text{temps mono} + \frac{\text{temps parallele}}{N}$$

Si S proportion sequentielle :

$$R = \frac{1}{S + \frac{(1-S)}{N}}$$

Si N tend vers l' ∞ (massivement parallèle)

$$R = \frac{1}{S}$$

R est inversement proportionel à la proportion sequentielle
quand N est très grand

Parallélisme explicite/implicite

- Explicite :
Cela a été programmé, lgges : MPI, OpenMP
- Implicite :
 - Plusieurs jobs en parallèle
 - Tirage Monte Carlo
 - Exploration paramétrique

Scale-UP

- SCALABILITE : capacité à supporter un plus grand nombre de tâches (identiques) en parallèle
- plus d'utilisateurs
- plus de transactions par minute
- plus de calculs (Threads) simultanés

SPEED-UP

- Capacité à exécuter une tâche en un temps plus court
- Si c'est par augmentation du nombre de CPU, le programme doit être parallélisable
- Calculs matriciels avec l'openMP, du MPI, ou des outils comme le fortran parallèle

Exemple Speed-up

- **Exploration paramétrique :**

Lancer n *application avec des paramètres d'entrée différents.

On peut le faire sur n machines en même temps au lieu de le faire en série.

- **Tirage Monte Carlo :**

1 Milliard d'expériences : durée T sur un coeur

Pas de dépendances (test sans mémoire)

Ces tests peuvent être lancés par script shell par exemple

Amélioration de la précision.

Codes et scalabilité

- Certains codes sont très scalables (exple Linpack)
- D'autres le sont beaucoup moins : soit à cause de la nature physique, soit à cause du découpage pas forcément adéquat au problème.
- Les échanges de données et la synchronisation prennent le pas sur les calculs

Workload-cache MISS/HIT

- l'espace occupé par une application (instr+data)="workload"
- Si application petite $\rightarrow$ tout est dans le cache
- Si application importante, une partie seulement dans le cache
- Miss ratio: nombre de fois sur 100 où la donnée n'est pas dans le cache
- Localité forte $\Rightarrow$ instruction miss ratio faible
- Les instructions actives sont souvent dans le cache $\rightarrow$ taux de miss instruction faible
- Les données restent rarement longtemps dans les caches: le tuning permettra de conserver une localité le plus possible.

Latence/Débit/Taux

- **Latence ou temps de réponse** : Durée entre la requête et la réponse de l'objet : latence mémoire, latence du réseau (durée du ping-pong/2), réponse à une requête base de données en transactionnel
- **Débit** : Quantité par unité de temps, débit mémoire, disque ou réseau en GB ou Gb/sec, MIPS, MFLOPS
- **Taux d'occupation ou d'utilisation d'une ressource** :
Mémoire, CPU...
Si saturé → Bottleneck ou chemin critique.
Système en entier (charge variable entre idle et 100%)

Autres métriques...

- **Efficacité** : rapport mesuré (théorique ou crête)
- **Fiabilité** : MTBE
- **Disponibilité** : Downtime/uptime, MTTF ...
- **Rapport performance/prix**
- **Performance/consommation energie** (Flops/watt)

xxx-bound

- **Code "CPU-bound"** : code dont la performance est limitée par la fréquence du CPU
- **Code "memory-bound"** : code limité par le débit mémoire: taux de miss élevé
- **Code "Latency-bound"** : code "memory-bound" pour lequel la latence des accès mémoires limite la performance
- **Code "IO-bound"** : code limité par le débit d'IO

IO-bound

- TRES fréquents mais ne le devrait pas en HPC saufs pour les nœuds d'E/S
- Sur les nœuds d'E/S (avec disques), IOstat donne des infos d'occupation disque.
Il faut paralléliser, stripper, SSD....ou changer d'architecture

100% CPU

- Memory bound ou CPU bound ?
- Processeurs modernes permettent de faire varier la fréquence (CPU governor)
 - Si performance suit la fréquence → CPU-bound
 - Si performance varie peu ou pas → memory-bound
- Extrapoler : avec un processeur plus rapide, je gagnerai ...
Attention aux extrapolations : abusives, linéaire ou pas, saturation/effet de seuil...

Considérations memory/CPU-bound

- Un calcul complexe même essentiellement CPU-bound sur un processeur, peut nécessiter un gros travail d'optimisation pour arriver à ce stade.
- La taille des jeux de données peut faire passer un code/calcul d'une catégorie à l'autre (effet cache)
- Un calcul peut être CPU-bound pour une famille de processeurs et memory-bound pour une autre.

Evaluation des métriques classiques

Débit mémoire

- Stream : développé initialement par John D. McCalpin

Copy : $c[i] = a[i];$

Scale : $c[i] = \text{scalar} * a[i];$

Add : $c[i] = a[i] + b[i];$

Triad : $c[i] = a[i] + \text{scalar} * b[i];$

Read pur : $\text{Tot} = \text{tot} + a[i]$

- Plusieurs tâches en parallèle pour saturer le débit mémoire
- Write = Read Modify write, donc peut compter pour 2
(depend de l'archi, du jeu d'instr.)

Evaluation des métriques classiques

- MFLOPS: oui et encore
Efficacité variable, proportion d'instructions flottantes dans l'appli
- MIPS = Meaningless Instructions per second néanmoins...
Utilisé pour des architectures très différentes

Obtenir les valeurs

- **Benchmarks ciblés pour des mesures élémentaires :**
Variantes de mesures mémoire, latence des caches et latence mémoire. Instrumentation HW pour le reste
- **Compteurs de performance :**
Jeu de registres HW enregistrant les événements,
Perfmon, Perfctr, PAPI (University Tennessee Knoxville),

Compteurs de performance

- Une quantité astronomique, mais peu en même temps (rejouer la même mesure plusieurs fois)
- Nombre de cycles (temps référence)
- Nombre cycles stalled : car on attend la mémoire ou autre ressource. Optimisation performance = chasse aux stalls
- Nombre instrs, nombre FP instrs.
- Nombre Load/store
- Nombre de branchement pris ou invalides
- Nombre de miss, taux de miss, nombre de miss/nombre instr , ...

Benchmarks et comparaison

- Une mesure isolée n'a pas de sens
Exemple:
la terre pèse $6 \times 10^{24} \text{ kg}$... et alors ?
Venus = 0.8 fois, Mars = 0.1 fois, Jupiter=315 fois...
- Une référence doit être définie car besoin d'explications, de conséquences ...